DondeAI Match, Recommendation & Ranking System

Version: V3.6 | **Last Updated:** February 2026 | **Status:** Production

Executive Summary

DondeAI is a restaurant recommendation engine for Chicago that combines algorithmic scoring with AI-generated narrative blurbs. When a user asks for a recommendation (e.g., "best sushi for a date night in Wicker Park, $$"), the system:

1. **Fetches** candidate restaurants from a Supabase database via RPC
2. **Classifies** the user's intent using Claude Haiku (cuisine, occasion, vibe, constraints)
3. **Scores** each candidate across five human-intuitive factors (0-10 each)
4. **Ranks** candidates using weighted composite scoring with power-law scaling (0-99 Donde Match)
5. **Sends** top candidates to Claude with live Google reviews for a personalized blurb
6. **Returns** the chosen restaurant with match score, factor breakdown, sub-component details, and recommendation text

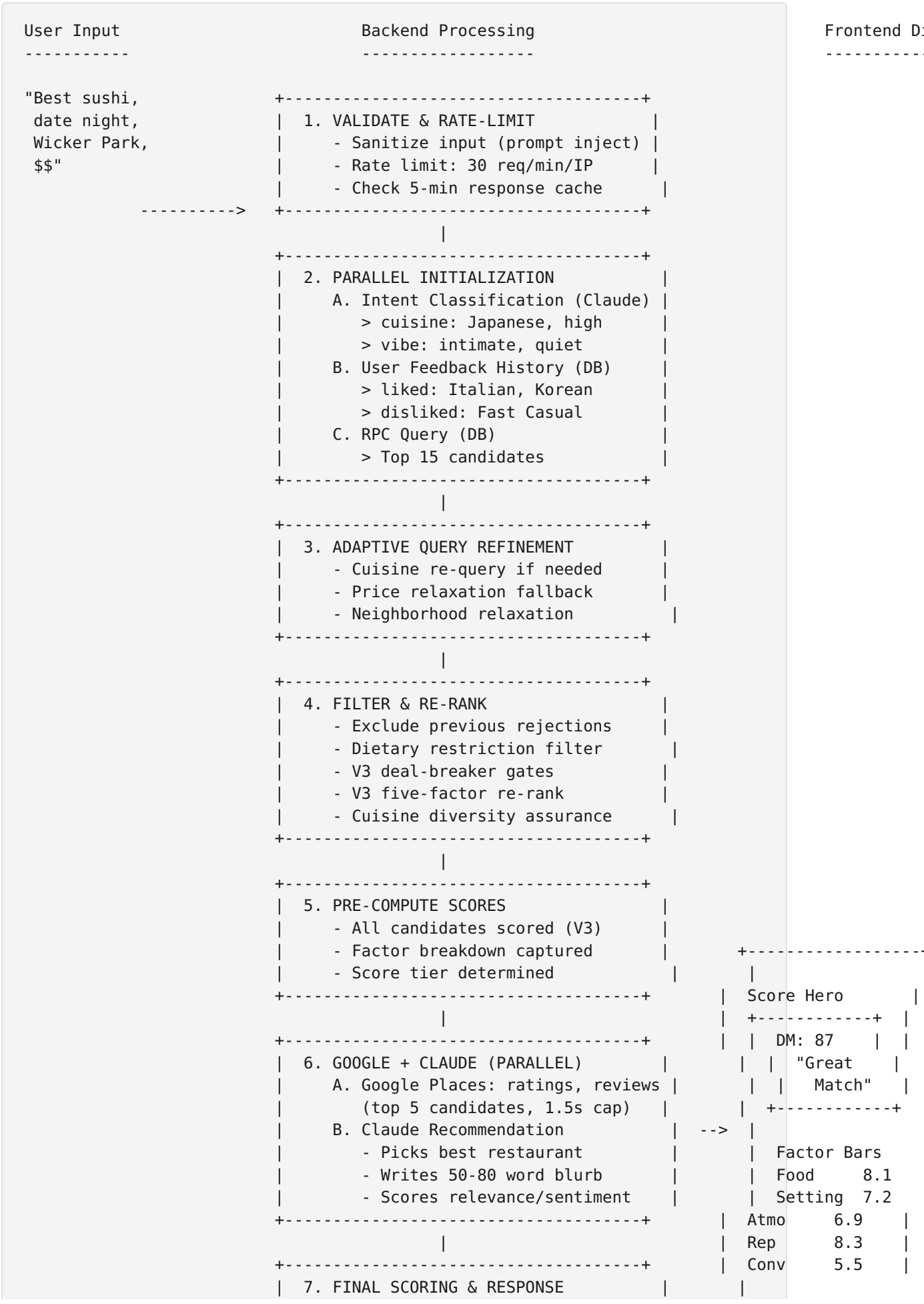
The system is designed around three principles: - **Transparency:** Users see exactly why a restaurant scored the way it did - **Honesty:** Low scores are acknowledged, not hidden. Trade-offs are named. - **Personalization:** Weights shift dynamically based on what the user actually asked for

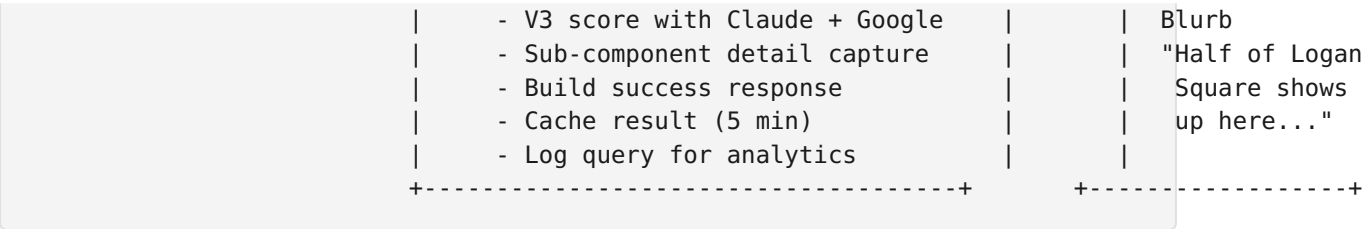
Table of Contents

---

1. End-to-End Process Flow

What happens when a user taps "Find a Spot"


User Input
Backend Processing
Frontend Di
"Best sushi, date night, Wicker Park, $$"
1. VALIDATE & RATE-LIMIT
- Sanitize input (prompt inject)
- Rate limit: 30 req/min/IP
- Check 5-min response cache
2. PARALLEL INITIALIZATION
A. Intent Classification (Claude)
> cuisine: Japanese, high
> vibe: intimate, quiet
B. User Feedback History (DB)
> liked: Italian, Korean
> disliked: Fast Casual
C. RPC Query (DB)
> Top 15 candidates
3. ADAPTIVE QUERY REFINEMENT
- Cuisine re-query if needed
- Price relaxation fallback
- Neighborhood relaxation
4. FILTER & RE-RANK
- Exclude previous rejections
- Dietary restriction filter
- V3 deal-breaker gates
- V3 five-factor re-rank
- Cuisine diversity assurance
5. PRE-COMPUTE SCORES
- All candidates scored (V3)
- Factor breakdown captured
- Score tier determined
6. GOOGLE + CLAUDE (PARALLEL)
A. Google Places: ratings, reviews (top 5 candidates, 1.5s cap)
B. Claude Recommendation
- Picks best restaurant
- Writes 50-80 word blurb
- Scores relevance/sentiment
7. FINAL SCORING & RESPONSE
Score Hero
DM: 87
"Great Match"
Factor Bars
Food 8.1
Setting 7.2
Atmo 6.9
Rep 8.3
Conv 5.5



Key Timing

Stage	Latency	Notes
Cache hit	<10ms	Returns instantly
Intent classification	200-400ms	Claude Haiku, runs in parallel
RPC database query	100-300ms	Runs in parallel
Google Places fetch	800-1500ms	Top 5 candidates, 1.5s timeout
Claude recommendation	400-800ms	Haiku 4.5, 512 tokens max
Total (cache miss)	**~1.5-2.5s**	Claude + Google dominate

---

2. The Five-Factor Scoring Model

The heart of DondeAI is a five-factor scoring system where each factor represents a distinct dimension of restaurant-user fit:

Factor	What It Measures	Scale	Key Data Sources
Food Match	Does the cuisine/menu match the craving?	0-10	Cuisine type, flavor profiles, dietary depth, signature dishes
Setting Fit	Does the environment suit the occasion?	0-10	Occasion scores, service style, meal pacing, social dynamics
Atmosphere	Does the vibe match what the user wants?	0-10	Noise, lighting, dress code, energy, music, decor
Reputation	Is this place well-regarded and trustworthy?	0-10	Google rating, sentiment analysis, awards, community standing
Convenience	Is it practical to go here right now?	0-10	Timing, reservations, wait time, parking, BYOB

Design Philosophy

> *"High score = best match. If nothing hits 80+, something is off."*

Each factor is independently scored 0-10, then combined via weighted average into a raw composite (0-10), which is transformed via power-law scaling into the final **Donde Match** score (0-99).

Why Five Factors?

Rationale: Expert research (behavioral psychology, UX design, food criticism) identified that diners evaluate restaurants along five mental dimensions whether they realize it or not:

1. "Will I like the food?" (Food Match)
2. "Is this the right place for this occasion?" (Setting Fit)
3. "Will the vibe be right?" (Atmosphere)
4. "Is it actually good?" (Reputation)
5. "Can I get there / get in?" (Convenience)

Collapsing these into a single score (like Yelp stars) loses information. Expanding beyond five creates cognitive overload. Five is the sweet spot.

---

3. Factor 1: Food Match (0-10)

Question: How well does this restaurant's food match what the user is craving?

Sub-Components

Sub-Component	Max Points	What It Measures
Cuisine Alignment	6.0	Exact cuisine match vs. related family vs. no match
Flavor Profile	2.0	Overlap between user's flavor preferences and restaurant's flavor profiles
Dietary Fit	2.0	How well dietary restrictions are accommodated
Menu Interest	1.0	Whether specific dishes mentioned in the request appear in the menu

Cuisine Alignment Scoring (0-6)

The most heavily weighted sub-component uses a tiered matching system:

Match Quality	Score	Example
Exact match	6.0	User asks "Japanese" and restaurant is Japanese
Close match	5.5	Subcategory or near-synonym
Subcategory match	5.0	User asks "Asian" and restaurant is Japanese
Cuisine family	3.5	User asks "Thai" and restaurant is Vietnamese (both Southeast Asian)
No match	0.0	User asks "Japanese" and restaurant is Mexican

Cuisine Families (used for family-match scoring): - Mediterranean: Greek, Italian, Middle Eastern - East Asian: Japanese, Chinese, Korean - Southeast Asian: Thai, Vietnamese - Latin American: Mexican, Peruvian, Brazilian, Puerto Rican - South Asian: Indian

Flavor Profile Matching (0-2)

Maps user flavor preferences to restaurant flavor profiles:

User Preference	Matches Profiles
"smoky"	smoky, charred, grilled, wood-fired
"spicy"	bold-spiced, chili-forward, fiery
"fresh"	bright-acidic, herbaceous, citrus-forward, light
"rich"	umami-forward, rich-buttery, creamy, decadent
"sweet"	sweet-savory, caramelized, honey-glazed
"tangy"	fermented, pickled, vinegar-bright
"earthy"	earthy, mushroom, truffle, root-vegetable
"savory"	umami-forward, savory, meaty

Scoring: Each matching flavor = +0.7 points, capped at 2.0.

Dietary Fit (0-2)

Restaurant's Dietary Depth	Score
Dedicated (e.g., fully vegan restaurant)	2.0
Solid (substantial options)	1.5

Restaurant's Dietary Depth	Score
Token (a few items)	0.5
No data, but all restrictions match options	1.0
Partial match (hierarchy, e.g., vegan at vegetarian)	0.5
No match	0.0

Adaptive Denominator (V3.6)

Problem: A restaurant with perfect cuisine match but no flavor data would score 6/11 = 5.5/10, unfairly low because it's being penalized for data we don't have.

Solution: The denominator only counts layers that have data:

```
maxPossible = 6 (cuisine, always present)
            + 2 (flavor, only if restaurant + user both have profiles)
            + 2 (dietary, always counted)
            + 1 (menu, only if signature dishes exist AND user mentioned specific food)

effectiveDenom = max(maxPossible, 8)   // Floor at 8 prevents over-inflation
normalized = min(10, score / effectiveDenom * 10)
```

Effect: Perfect cuisine match with no flavor/menu data: 6.5/8 * 10 = **8.1/10** (was 5.9/10 before V3.6).

---

4. Factor 2: Setting Fit (0-10)

Question: Does this restaurant's physical environment and service style match the occasion?

Sub-Components

Sub-Component	Max Points	What It Measures
Occasion Base	7.0	DB occasion scores (date_friendly, group_friendly, etc.) power-stretched
Service Style	+1.5 / -0.5	Whether the service style fits (omakase for date night vs. counter for business)
Social Dynamics	+1.5	Meal pacing, kid-friendliness, conversation fit, group size

Occasion Base Scoring

Each restaurant has seven occasion scores in the database (0-10 each): - `date_friendly_score`, `group_friendly_score`, `family_friendly_score` - `business_lunch_score`, `solo_dining_score`, `romantic_rating`, `hole_in_wall_factor`

These are blended using occasion-specific weights:

Occasion	Score Blend
Date Night	100% date_friendly_score
Group Hangout	100% group_friendly_score
Special Occasion	70% romantic_rating + 30% date_friendly_score
Adventure	60% hole_in_wall + 20% group + 20% solo
Treat Myself	50% solo + 30% romantic + 20% hole_in_wall
Any	Average of all 7 scores

The blended score is then power-stretched: `pow(base/10, 0.85) * 7` to widen the discriminating range.

Service Style Matching

Expert-curated lookup tables match service styles to occasions:

Occasion	Good Fit (+1.5)	Bad Fit (-0.5)
Date Night	Full Table, Omakase, Tasting Menu, Bar	Fast Casual
Business Lunch	Full Table	Counter, Fast Casual
Group Hangout	Full Table, Family Style, Fast Casual, Bar	Omakase
Special Occasion	Tasting Menu, Omakase, Full Table	Fast Casual, Counter
Solo Dining	Counter, Bar, Fast Casual, Full Table	(none)

Rationale (behavioral psychology): Service style creates the frame for the entire dining experience. A tasting menu on a first date signals intention. Counter service at a business lunch signals "you're not worth a reservation."

---

5. Factor 3: Atmosphere (0-10)

Question: Will the vibe feel right for what the user wants?

This is the most complex factor with the most sub-components, because "vibe" is multi-dimensional.

Sub-Components

Sub-Component	Max Points	What It Measures
Noise Level	2.0	Does the noise level match the occasion?
Lighting	2.0	Does the lighting match the occasion?
Dress Code	1.0	Is the dress code appropriate?
Energy Level	2.0	Does the energy level match the occasion?
Music Vibe	1.5	Does the music style fit?
Vibe Keywords	1.5	Do user-requested vibes (cozy, lively) match?
Conditional Features	Up to 5.0	Live music, outdoor, scenic view, seasonal, Instagram-worthy

Occasion Noise Expectations

Occasion	Acceptable Noise
Date Night	Quiet, Moderate
Business Lunch	Quiet
Group Hangout	Moderate, Loud
Family Dinner	Quiet, Moderate
Adventure	Quiet, Moderate, Loud
Solo Dining	Quiet, Moderate

Occasion Energy Ranges (0-10 scale)

Occasion	Energy Range	Sweet Spot
Date Night	4-7	5.5
Group Hangout	6-9	7.5
Business Lunch	2-5	3.5
Family Dinner	3-6	4.5
Adventure	4-10	7.0

Occasion	Energy Range	Sweet Spot
Chill Hangout	3-6	4.5

Scoring: If the restaurant's energy falls within the range = 2.0 points. Otherwise: `2.0 - (distance * 0.4)`.

Conditional Features (Request-Driven)

These sub-components only activate when the user explicitly requests them:

Feature	Max	Trigger Keywords
Live music	1.5	"live music," "live jazz," "live band"
Music style	1.0	"jazz," "acoustic," "blues"
Outdoor seating	1.0	"outdoor," "patio," "al fresco"
Scenic view	1.0	"view," "rooftop," "skyline"
Seasonal	0.5	Restaurant's seasonal relevance >= 7 for current season
Instagram-worthy	1.0	"instagram," "aesthetic," "photogenic"

Adaptive Denominator (V3.6)

Same principle as Food Match: only layers with actual data count in the denominator.

```
atmoMaxPossible starts at 0
  + 2.0 if restaurant has noise_level data
  + 2.0 if restaurant has lighting data
  + 1.0 if restaurant has dress_code data
  + 2.0 if restaurant has energy_level data
  + 1.5 if restaurant has music_vibe data
  + conditional features (only if user requested AND restaurant has data)

effectiveMax = max(atmoMaxPossible, 5.0)   // Floor prevents over-inflation
```

Cold start (no data at all): Returns 4.0 neutral score. No Bayesian gating applied.

---

6. Factor 4: Reputation (0-10)

Question: Is this place well-regarded? Can the user trust it?

Sub-Components

Sub-Component	Max Points	What It Measures
Google Rating	5.0	Stretched rating (3.0-5.0 to 0-5.0) weighted by review count confidence
Sentiment	2.0	AI-analyzed review sentiment with negative review penalty
Awards	2.0	Awards, notable chef, cultural authenticity
Community	2.0	Neighborhood integration (institution, destination, trending)

Google Rating Stretch (V3.6)

Problem: Google ratings cluster between 3.5-5.0. A raw linear map wastes half the scale.

Solution: Stretch the meaningful range:

```
normalized = max(0, (rating - 3.0) * 2.5)
score = min(5, normalized * confidence)
```

Confidence levels (based on review count):

Reviews	Confidence	Rationale
200+	1.0	Statistically reliable
50-199	0.9	Good signal
10-49	0.8	Moderate signal
<10	0.7	Sparse, discount slightly

Effect: A 4.5-star restaurant with 200+ reviews: (4.5 - 3.0) * 2.5 * 1.0 = **3.75/5** (was 2.67/4 in V3.5).

Awards and Community Scoring

Signal	Points
Has awards_recognition	+1.0
Has chef_notable	+0.5
Cultural authenticity >= 8	+0.5
Neighborhood institution	+1.5
Destination venue	+1.0

Signal	Points
Hidden local favorite	+0.5
Trending score >= 7	+0.5

Neutral Defaults (when data is missing)

Sub-Component	Default	Rationale
Google	2.5	Conservative prior: no rating doesn't mean bad
Sentiment	1.0	No reviews = unknown, not negative
Awards	0.5	Most restaurants don't have awards data
Community	0.5	Most restaurants don't have integration data

Expert input (behavioral psychology): Absence of evidence is not evidence of absence. Missing Google data should pull toward a neutral prior, not zero. Users would find it unfair if a new restaurant scored 0 on reputation just because it hasn't been reviewed yet.

---

7. Factor 5: Convenience (0-10)

Question: How practical is it to go here right now?

Sub-Components

Sub-Component	Points	What It Measures
Timing Fit	-2 to +2	Does the restaurant's best time match the user's time of day?
Reservation	-2.5 to +2	Ease of getting in (walk-in friendly vs. hard to book)
Wait Time	-1 to +1	Expected wait duration
Practical	-0.5 to +1.5	Cash-only penalty, BYOB match, parking

Base score: Starts at 4.0 (neutral starting point).

Spontaneity Detection

The system detects spontaneous dining intent from request text: - Keywords: "tonight," "right now," "last minute," "walk-in," "spontaneous" - Also from V2 intent: `spontaneity: "spontaneous"`

Impact: When spontaneous, a hard-to-get reservation is penalized more heavily (-2.5 vs -1.0).

---

8. Dynamic Weight System

Weights determine how much each factor contributes to the final score. They shift dynamically based on **what the user is looking for**.

Base Weights (Default)

```
Food: 30%  |  Setting: 25%  |  Atmosphere: 20%  |  Reputation: 15%  |  Convenience: 10%
```

Cuisine Importance Override

When a user specifies a strong food preference, weights shift toward Food Match:

Cuisine Importance	Food	Setting	Atmosphere	Reputation	Convenience
High ("best sushi")	45%	15%	15%	15%	10%
Medium ("maybe Italian")	35%	20%	20%	15%	10%
Low (no food preference)	15%	20%	30%	15%	20%

Occasion Override

Each occasion has its own weight distribution:

Occasion	Food	Setting	Atmosphere	Reputation	Convenience
Date Night	20%	30%	25%	15%	10%
Group Hangout	30%	25%	20%	15%	10%
Family Dinner	25%	25%	15%	15%	20%
Business Lunch	20%	30%	25%	15%	10%
Solo Dining	30%	15%	20%	15%	20%
Adventure	20%	25%	15%	25%	15%
Special Occasion	20%	30%	25%	15%	10%
Treat Myself	30%	15%	25%	20%	10%
Chill Hangout	20%	20%	25%	10%	25%

Cuisine x Occasion Blending (V3.6)

When both cuisine importance and occasion are specified, they blend:

```
cuisineBlend = high ? 0.70 : medium ? 0.40 : 0.00
final_weight = cuisine_weight * cuisineBlend + occasion_weight * (1 - cuisineBlend)
```

Example: "best sushi, date night" - Cuisine importance = high, so cuisineBlend = 0.70 - Food weight = 0.45 * 0.70 + 0.20 * 0.30 = **0.375** (37.5%) - Setting weight = 0.15 * 0.70 + 0.30 * 0.30 = **0.195** (19.5%) - Atmosphere weight = 0.15 * 0.70 + 0.25 * 0.30 = **0.180** (18.0%)

Rationale (V3.6 fix): Previously, `cuisine_importance === "high"` completely skipped occasion blending, which meant "best sushi for a date night" ignored date night weights entirely. The graduated blend ensures both dimensions are respected proportionally.

Emotional Intent Fine-Tuning

V2 intent classification also nudges weights:

Emotional Intent	Adjustment
Explore	+5% reputation, -5% food
Comfort	+5% atmosphere, -5% reputation
Impress	+5% reputation, -5% convenience

---

9. From Raw Composite to Donde Match (0-99)

Step-by-Step Pipeline

```
Step 1:  Compute 5 factor scores (0-10 each)
Step 2:  Apply Bayesian gating to enrichment-dependent factors
Step 3:  Apply decorrelation discounts
Step 4:  Weighted composite = sum(factor * weight)  -> raw (0-10)
Step 5:  Quality match bonus (+0 to +0.8)
Step 6:  Claude relevance modulation (+/- 0.5)
Step 7:  Deal-breaker penalties (subtractive)
Step 8:  Personalization adjustments
Step 9:  Clamp: max(0, raw)
Step 10: Power-law scaling -> Donde Match (0-99)
```

Bayesian Gating (Step 2)

Problem: Some factors rely on enrichment data (deep profiles generated by Claude). If enrichment confidence is low, those factor scores might be unreliable.

Solution: Shrink uncertain scores toward a population prior:

```
PRIOR_MEAN = 5.5
GATING_THRESHOLD = 3  (confidence below 3 triggers gating)
shrinkageWeight = enrichment_confidence / 6

gated_score = PRIOR_MEAN * (1 - shrinkageWeight) + raw_score * shrinkageWeight
```

Gated factors: Food Match, Setting Fit, Atmosphere (depend on deep profiles)

NOT gated: Reputation (Google data), Convenience (DB data) -- independent sources

Expert input (statistics): This is Bayesian shrinkage, the same principle used in baseball batting averages. A player with 2 at-bats hitting 1.000 is pulled toward the league average. Similarly, a restaurant with low-confidence data is pulled toward a neutral score rather than being rewarded or punished for unreliable data.

Decorrelation Discounts (Step 3)

Problem: Some factors are correlated. A restaurant with great food tends to have great reputation. Counting both at full value double-counts quality.

Solution:

```
// Setting/Atmosphere overlap (both measure "vibe")
if (setting > 7 AND atmosphere > 7):
  overlap = min(setting - 7, atmosphere - 7) * 0.10
  atmosphere -= overlap

// Food/Reputation overlap (both measure "quality")
if (food > 7 AND reputation > 7):
  overlap = min(food - 7, reputation - 7) * 0.05
  reputation -= overlap
```

The discount only applies to the excess above 7 and uses small coefficients (0.10, 0.05). This prevents double-counting while preserving most of the signal.

Quality Match Bonus (Step 5)

Rewards restaurants that are excellent across multiple dimensions:

Condition	Bonus
3+ factors >= 6.5	+0.5 (multi-factor excellence)
All factors >= 5.0	+0.3 (well-rounded)
Any factor >= 8.0	+0.3 (strong lead)
Cap	**+0.8 max**

Power-Law Scaling (Step 10)

The final transformation from raw composite (0-10) to Donde Match (0-99):

```
rawNormalized = max(0, min(1, raw / 10))
scaled = pow(rawNormalized, 0.73)
dondeMatch = min(99, max(0, round(scaled * 116)))
```

Why power-law?

Raw composites cluster in a narrow range (~3.5-7.5 out of 10) because factors use conservative defaults and perfect 10s are nearly impossible. A linear map would compress most restaurants into 35-75 on a 0-99 scale. The power-law stretches this:

Raw Composite	Linear (*10)	Power-Law (0.73, *116)
3.0	30	47
5.0	50	68
6.5	65	81
7.5	75	89
8.5	85	96

Expert input (UX psychology): Users expect scores to work like school grades: 90+ is excellent, 70-80 is good, below 60 is concerning. The power-law aligns the mathematical output with human expectation.

---

10. Deal-Breaker Gates and Penalties

Pre-Scoring Gates

Before any scoring happens, candidates are filtered:

Gate	Action	Rationale
Excluded restaurant ID	Remove	User already rejected this
No dietary options at all when user has restrictions	Remove	Absolute deal-breaker

Post-Scoring Penalties (Subtractive, on Composite)

Penalty	Value	Trigger
Dietary incompatibility	-0.8 to -2.5	Restaurant can't accommodate dietary needs

Penalty	Value	Trigger
Price over-budget (1 tier)	-0.5	$$ budget, $$$ restaurant
Price over-budget (2 tiers)	-1.5	$$ budget, $$$$ restaurant
Price over-budget (3 tiers)	-3.0	$ budget, $$$$ restaurant
Neighborhood mismatch	-0.6	Specific neighborhood requested, restaurant is elsewhere

Key design decisions: - **No "under-budget" penalty:** Budgets are ceilings, not targets. A $$ restaurant for a $$$ budget is fine. - **Dietary incompatibility uses a hierarchy:** Vegan at a vegetarian restaurant gets a softer penalty (-0.8) than vegan at a steakhouse (-2.5). - **Neighborhood penalty is small (0.6)** relative to price (0.5-3.0): Expert research showed price mismatches cause more regret than location mismatches (2.5:1 ratio from behavioral economics research on "mental accounting").

---

11. Personalization and Feedback Loops

User Feedback Signals

When users tap "like" or "dislike" on a recommendation, it feeds back into future scoring:

Signal	Adjustment	Rationale
Liked cuisine	+1.0	Positive reinforcement for familiar cuisines
Disliked cuisine	-1.0	Explicit negative signal
Disliked specific restaurant	-2.0	Strong: user rejected this exact place

Asymmetry (Prospect Theory): Dislikes are weighted roughly 2x likes. Research shows losses (bad dining experiences) are felt more strongly than gains (good ones). The 2:1 ratio matches Kahneman and Tversky's loss aversion coefficient.

Rejection Pattern Analysis

When a user taps "Try Another" 2+ times, the system analyzes rejected restaurants:

```
rejectionSignals = {
  avoidCuisines: ["Italian", "Mexican"],    // Cuisines of rejected restaurants
  avoidPriceLevels: ["$$$"],                // Price levels of rejected restaurants
}
```

These signals penalize similar candidates: - Avoid cuisine: -0.7 (inferred signal, softer than explicit dislike) - Avoid price level: -1.5 (only if not already penalized by deal-breaker)

Stacking prevention: If a restaurant is already penalized for explicit cuisine dislike (-1.0), the inferred avoidCuisine penalty (-0.7) is skipped to prevent double-punishment.

"Try Again" Flow

When a user rejects a recommendation:

1. Frontend sends same request + `exclude: [rejected_restaurant_id]`
2. Backend skips cache (ensures fresh results)
3. Rejected restaurants are filtered out
4. Rejection patterns are analyzed and fed to both scoring and Claude
5. Claude receives context: "The user has rejected 2 previous suggestions. They seem to want something different from Italian and $$$. Prioritize variety."

---

12. The Claude Recommendation Engine

How Claude Is Used

Claude serves two roles in the pipeline:

1. **Intent Classification** (Claude Haiku, at the start) -- Understands what the user wants
2. **Recommendation Writing** (Claude Haiku, after scoring) -- Writes the personalized blurb

Intent Classification

A separate Claude call classifies the user's natural language request:

Input: "best sushi for a date night, somewhere quiet"

Output:

```
{
  "target_cuisines": ["Japanese"],
  "cuisine_importance": "high",
  "vibe_keywords": ["quiet", "intimate"],
  "emotional_intent": "impress",
  "date_type": "date_night",
  "spontaneity": "planned",
  "flavor_preferences": ["fresh", "savory"],
  "practical_constraints": []
}
```

This informs weight selection, food matching, and atmosphere scoring.

Recommendation Prompt Architecture

System Prompt (~2500 words, cached for 5 minutes): - Persona: "You are Donde, a sharp, opinionated Chicago dining guide" - Voice: Write like a food-obsessed friend texting about where to eat - Cultural grounding: Use correct cuisine vocabulary (say "mole negro" not "dark sauce") - Writing rules: 50-80 words, one sensory detail, one honest caveat, zero AI slop - Tone modulation: Calibrate confidence to match score tier (HIGH/MID/LOW) - 30+ banned AI slop patterns (culinary, gastronomic, unforgettable, etc.) - Strict JSON output schema

User Prompt (~500-2000 words, dynamic): - User request (occasion, budget, neighborhood, special request, dietary restrictions) - Top 10 candidates with full metadata, deep profiles, and Google reviews - Preliminary Donde Match score per candidate (DM:87) - Factor scores per candidate (F:8.1/7.2/6.9/8.3/5.5) - Rejection context (if "Try Again") - Cuisine mismatch context (if cuisine unavailable)

Voice Modulation

The system prompt shifts personality based on occasion:

Occasion	Voice
Adventure	"Street-smart Chicago food explorer. Found this place by accident, can't stop going back."
Date Night	"Quietly confident. Like a friend who's been on enough dates to know what works."
Business Lunch	"Efficient, credible, no-frills. Sound like a colleague who knows the good spots near the office."
Group Hangout	"The friend who always picks the right dinner spot. Energetic, practical, fun."
Special Occasion	"Confident and warm with a touch of polish. Like a friend who knows wine and can get you a table."
Solo Dining	"Gentle, knowing, appreciative. Like a friend who understands the joy of eating alone well."
Family Dinner	"Warm and practical. Like a parent-friend who knows which restaurants work with kids AND adults."
Chill Hangout	"Low-key, easy, no pressure. The friend who knows the perfect spot where nobody needs an agenda."

Tone Modulation (V3.5)

The blurb's confidence level is calibrated to match the score:

Score Tier	Tone
	Full confidence. Declarative. "This is the one." Caveat goes last, minor qualifier.

Score Tier	Tone
HIGH (DM 80+)	
MID (DM 55-79)	Open with what works, name honest trade-off, close on user-relevant strength. Never apologize.
LOW (DM <55)	Lead with strongest genuine positive. Brief gap acknowledgment. End with actionable tip.

Rationale (behavioral psychology): When a user sees 87, the blurb should feel confident. When they see 62, the blurb should feel honest. Misalignment between score and tone erodes trust.

Factor Score Injection (V3.6)

Factor scores are injected per candidate so Claude can emphasize strengths and acknowledge weaknesses:

```
F:8.1/7.2/6.9/8.3/5.5  ->  Food/Setting/Atmo/Rep/Conv
```

Claude receives the instruction: "Lead with the restaurant's strongest factor(s). If one factor is notably weak (<4), briefly acknowledge the trade-off rather than pretending it's perfect."

Quality Guardrails

After Claude responds, the system checks for:

Check	Pattern	Action
AI slop	30+ banned words/phrases	Warn (log)
Em dashes	Unicode em-dash character	Warn (log)
Structural tells	"Ah,", "Whether...or...", "If you're looking for..."	Warn (log)
Word count	> 100 words	Warn (log)

Fallback Chain

If Claude fails entirely, the system degrades gracefully:

```
Claude JSON parse -> Regex recovery -> Template response -> One-liner -> Generic fallback
```

Template responses use occasion-specific openers and deep profile data to generate adequate (but less personalized) blurbs without any AI call.

---

13. The UI: Score Display and Drill-Down

Progressive Disclosure (3 Levels)

The UI reveals scoring information progressively:

Level 1: Score Hero - Large semicircular arc gauge showing Donde Match (0-99) - Animated count-up from 0 to final score (1.5s duration) - Arc color transitions: red to amber to green as score climbs - Verdict label: "Outstanding" (90+), "Great Match" (80-89), "Good Pick" (70-79), "Worth a Try" (55-69)

Level 2: Factor Bars (tap to reveal) - Five horizontal bars showing factor scores - Color-coded: Accent (>=7.5), Gray (5-7.4), Amber (<5) - Dominant factor (highest score) highlighted with thicker bar - Staggered animation: each bar slides in 60ms after the previous

Level 3: Inline Explanation Cards (tap factor bar to reveal) - Sub-component breakdown per factor - Mini progress bars showing score/max per sub-component - Signal text explaining what contributed - Weight percentage footer ("Weight in your search: 25%")

Score Verdict Tiers

Score	Verdict	Arc Color
90-99	Outstanding	Green
80-89	Great Match	Green
70-79	Good Pick	Amber
55-69	Worth a Try	Amber
40-54	It's a Stretch	Red
0-39	Weak Match	Red

Sub-Component Mini-Bars

Each sub-component shows:

```
+--------------------------------------------------+
|  Cuisine    ========..   6/6.5   Strong match    |
|  Flavor     ===.......   1/2     Partial fit     |
|  Dietary    =====.....   0.5/2   Token options   |
|  Menu       ..........   0/1     No match        |
|  ----------------------------------------       |
|  Weight in your search                   38%     |
+--------------------------------------------------+
```

Sub-Component Label Map

Factor	Sub-Components
Food Match	Cuisine, Flavor, Dietary, Menu Match
Setting Fit	Occasion Fit, Service, Social Fit
Atmosphere	Noise, Lighting, Dress Code, Energy, Music
Reputation	Google Rating, Reviews, Awards, Community
Convenience	Timing, Reservations, Practical

---

14. Optimization History

The scoring engine has gone through 7 expert review cycles, each informed by different domain experts:

V3.1 -- Foundation (Expert Review Cycle 1)

- Introduced power-law scaling (exponent 0.85) to stretch compressed score range
- Reduced neutral defaults to lower score floor from ~37 to ~25 DM
- Added Bayesian shrinkage for enrichment confidence gating
- Stretched Google rating to actual distribution (3.5-5.0)
- Applied Claude relevance to composite (not individual factors) to preserve display integrity
- Removed under-budget price penalty (budgets are ceilings)

V3.2 -- Calibration (Expert Review Cycle 2)

- Raised scale multiplier 99 to 105 to make 90+ DM reachable
- Scoped Bayesian gating to enrichment-dependent factors only
- Atmosphere cold-start: 3.5 neutral when zero data
- Single penalty clamp: removed intermediate max(0) from penalty pipeline
- Rejection signal stacking prevention
- Neighborhood penalty reduced -1.0 to -0.6 (behavioral economics: 2.5:1 ratio vs price)
- Adventure weights rebalanced: setting 0.15 to 0.25

V3.3 -- Edge Cases (Expert Review Cycle 3)

- Fixed Food maxPossible 10 to 11 (Layer 1 was expanded to 0-6 but denominator was stale)
- Atmosphere cold-start bypasses Bayesian gating (gating would invert the score)
- Price stacking prevention: skip inferred avoidPrice if deal-breaker already applied

- Added missing occasion weights: Solo Dining, Treat Myself, Chill Hangout
- Fixed avoidCuisine penalty: -2.0 to -0.7 (inferred signal should be softer than explicit)

V3.4 -- Distribution Stretch (Expert Review Cycles 4-5)

- Power-law exponent 0.85 to 0.73, multiplier 105 to 116 (widens practical score range)
- Added quality match bonus: +0.5 for 3+ strong factors, +0.3 well-rounded, +0.3 strong lead
- Bayesian prior 5.0 to 5.5, threshold 5 to 3 (less aggressive gating)
- Liked cuisine bonus 0.5 to 1.0 (rebalance like/dislike ratio)
- Decorrelation coefficients softened: 0.15 to 0.10, 0.10 to 0.05

V3.5 -- Tone Modulation (Expert Review Cycle 6: CW/UX/BP/FC)

- Pre-compute preliminary DM before Claude call
- Inject score-tier tone directive into system prompt
- Show preliminary DM per candidate in user prompt for tone-aware writing
- Experts consulted: Copywriter, UX Writer, Behavioral Psychologist, Food Critic

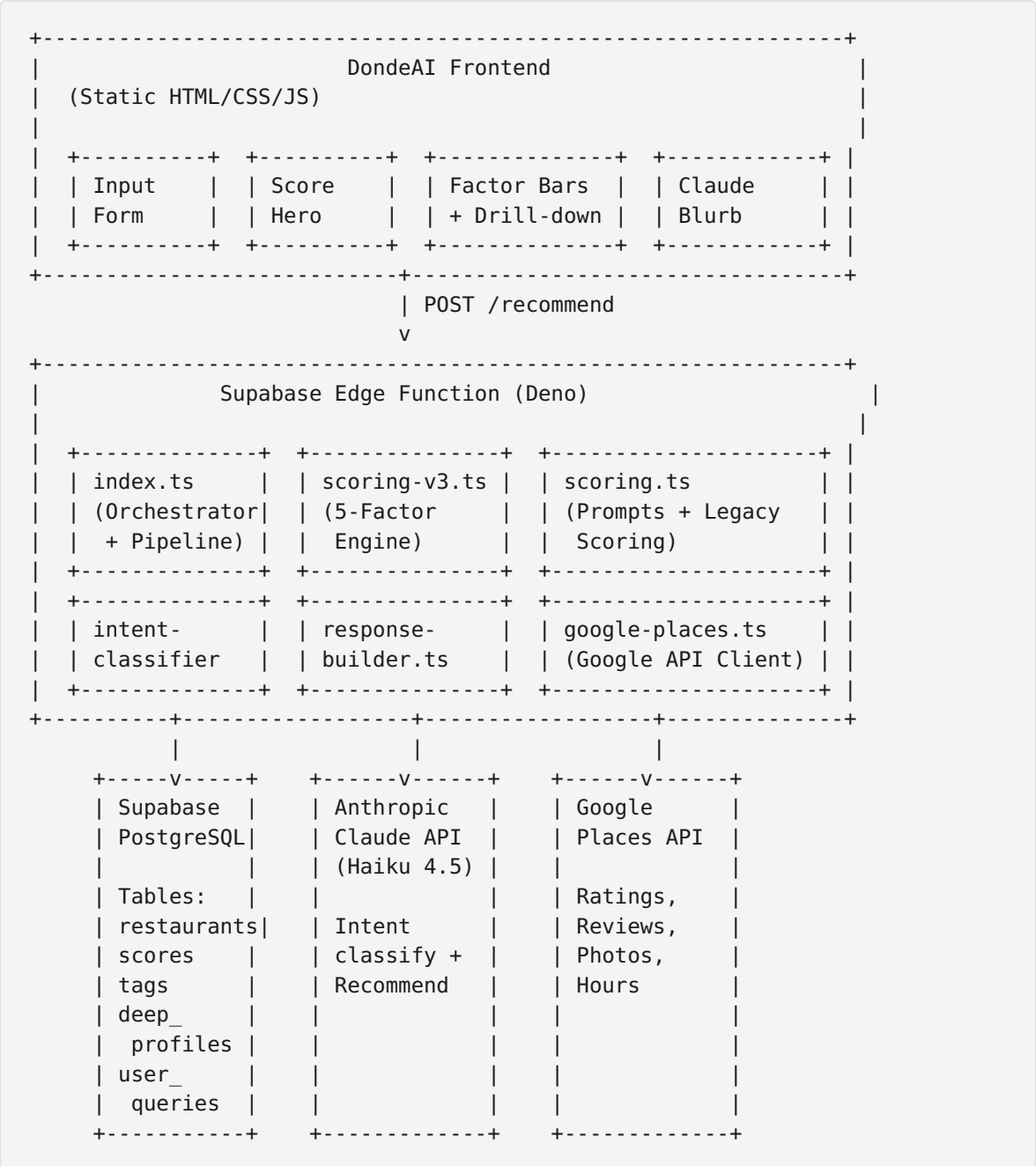
V3.6 -- Transparency and Recalibration (Current)

- Sub-component detail tracking (V3SubComponent) for UI drill-down
- Adaptive denominators: only count layers with actual data
- Food Match: effective denominator drops to 8 when flavor/menu data absent
- Atmosphere: only active layers counted (floor at 5.0)
- Google rating stretch widened: 3.5-5.0 to 3.0-5.0 with ceiling raised 4 to 5
- Cold-start neutral raised: 3.5 to 4.0
- Factor scores injected into Claude prompt for blurb emphasis
- Cuisine x Occasion weight blending (graduated ratios instead of binary skip)
- Factor color threshold: 7 to 7.5 for accent color

---

15. Architecture and Data Flow Diagram

System Architecture



Data Sources Per Factor

Factor	Database Fields	Deep Profile Fields	External APIs	User Input
Food Match	cuisine_type, dietary_options, tags	flavor_profiles, signature_dishes, dietary_depth, cuisine_subcategory	(none)	target_cuisines, flavor_preferences, dietary_restrictions

Factor	Database Fields	Deep Profile Fields	External APIs	User Input
Setting Fit	occasion scores (7 columns)	service_style, meal_pacing, kid_friendliness, conversation_friendliness, group_size_sweet_spot, date_progression	(none)	occasion, date_type, group_size_hint
Atmosphere	noise_level, lighting_ambiance, dress_code, outdoor_seating, live_music	energy_level, music_vibe, decor_style, instagram_worthiness, seasonal_relevance	(none)	vibe_keywords, special_request keywords
Reputation	trending_score	awards_recognition, chef_notable, cultural_authenticity, neighborhood_integration	Google rating, review count, sentiment analysis	(none)
Convenience	best_times, parking_availability	reservation_difficulty, typical_wait_minutes, byob_policy, payment_notes	(none)	time_of_day, spontaneity

16. Glossary

Term	Definition
Donde Match (DM)	Final score 0-99 shown to users. Combines all five factors via power-law scaling.
Factor	One of five scoring dimensions: Food Match, Setting Fit, Atmosphere, Reputation, Convenience. Each 0-10.
Sub-Component	A constituent part of a factor. E.g., Food Match has Cuisine, Flavor, Dietary, Menu.
Weight	The proportion (0.0-1.0) determining how much a factor contributes to the composite. Sums to 1.0.
Adaptive Denominator	Normalization technique that only counts data layers with actual data, preventing absent data from deflating scores.
Bayesian Gating	Shrinkage of uncertain factor scores toward a population prior (5.5). Applied when enrichment confidence is low.

Term	Definition
Power-Law Scaling	Non-linear transformation (exponent 0.73, multiplier 116) that stretches the compressed raw composite range to fill 0-99.
Deep Profile	AI-enriched restaurant metadata (30+ fields): signature dishes, service style, energy level, origin story, etc.
Enrichment Confidence	0-10 score indicating how reliable the deep profile data is.
Decorrelation	Small discount applied to correlated factors (setting/atmosphere, food/reputation) to prevent double-counting.
Quality Bonus	Reward for restaurants scoring well across multiple factors (+0.8 max).
Deal-Breaker Gate	Hard filter removing candidates before scoring (excluded IDs, dietary incompatibility).
Rejection Signals	Inferred preferences from "Try Another" patterns (avoided cuisines, price levels).
Tone Modulation	System calibrating Claude's blurb confidence to match the Donde Match score tier.
Factor Legend	Compact factor score tag injected per candidate in Claude's prompt (F: 8.1/7.2/6.9/8.3/5.5).
RPC	Remote Procedure Call: Supabase database function returning pre-ranked restaurant candidates.
Slop Detection	Quality guardrail flagging AI-generated text with banned patterns.

This document describes the DondeAI recommendation system as of V3.6. For the source code, see: - Scoring Engine: `dondeBackend/supabase/functions/recommend/_shared/scoring-v3.ts` *- Prompts:* `dondeBackend/supabase/functions/recommend/_shared/scoring.ts` *- Response Builder:* `dondeBackend/supabase/functions/recommend/_shared/response-builder.ts` *- Orchestrator:* `dondeBackend/supabase/functions/recommend/index.ts` *- UI:* `dondeAI/js/animations.js`, `dondeAI/js/utils.js`, `dondeAI/css/components.css`